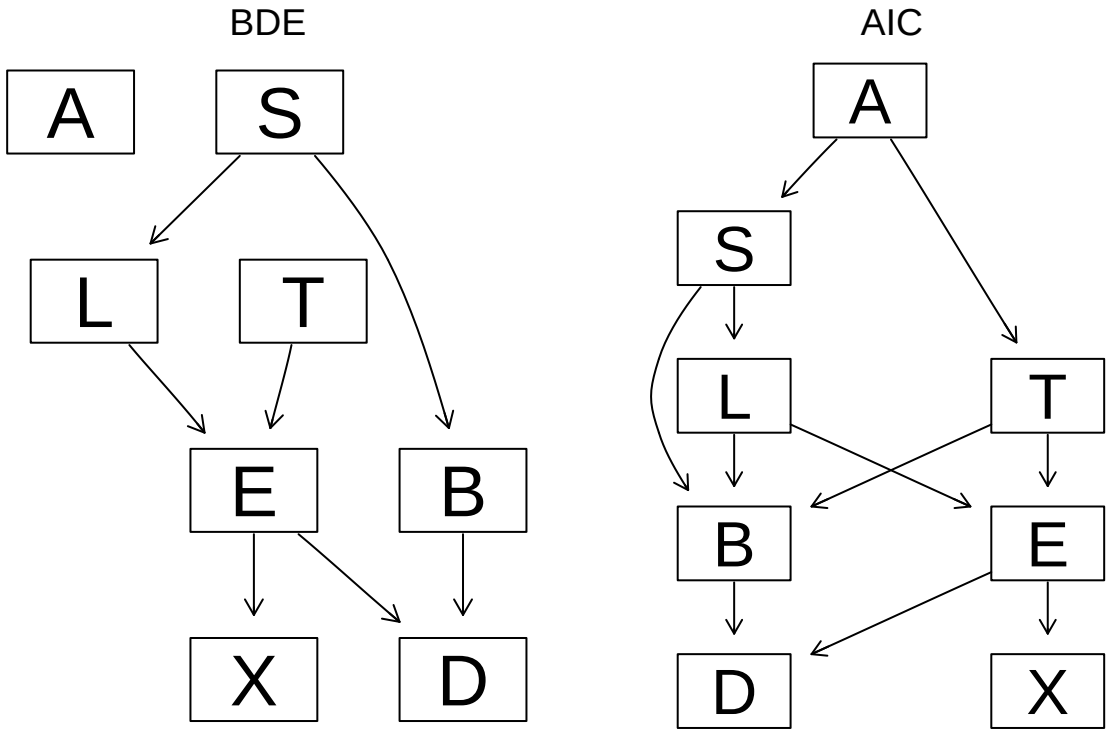
Lab 1: Graphical Model

Manfred Clase (mancl774)

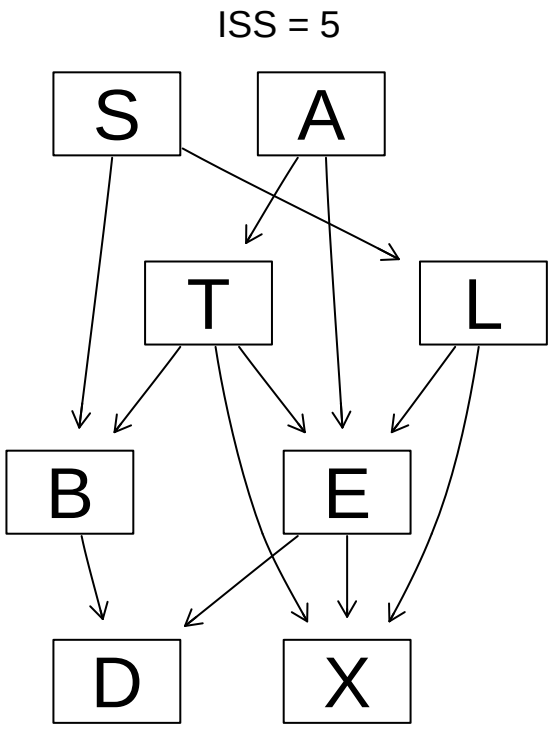
2026-09-13

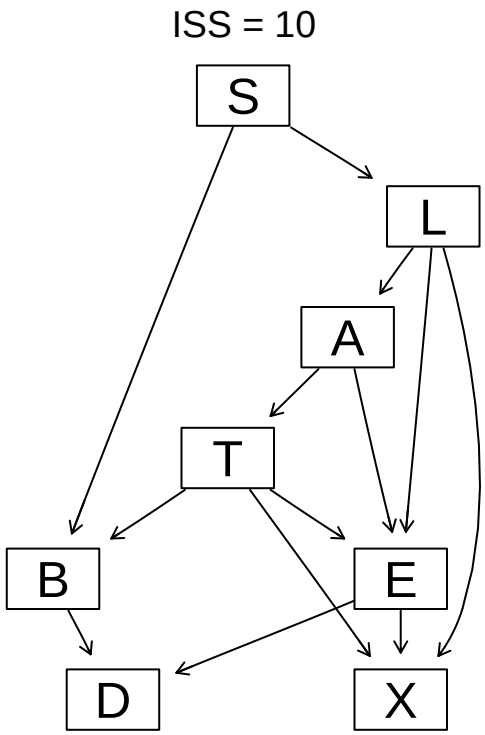
Question 1

Show that multiple runs of the hill-climbing algorithm can return non-equivalent Bayesian network (BN) structures. Explain why this happens.

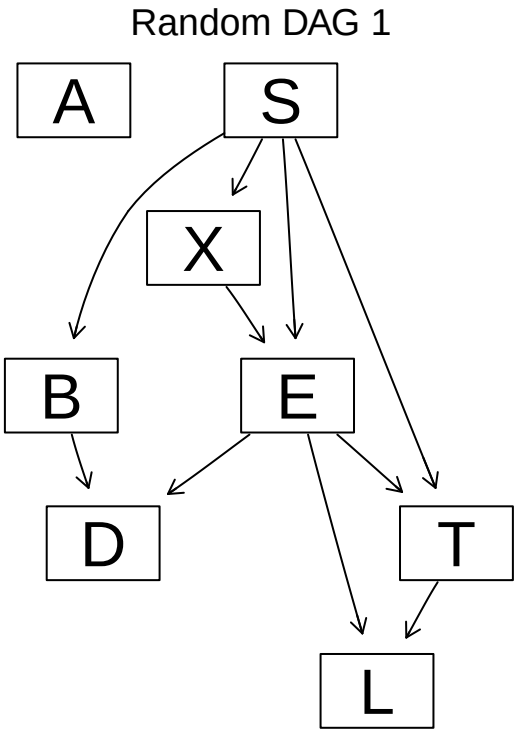


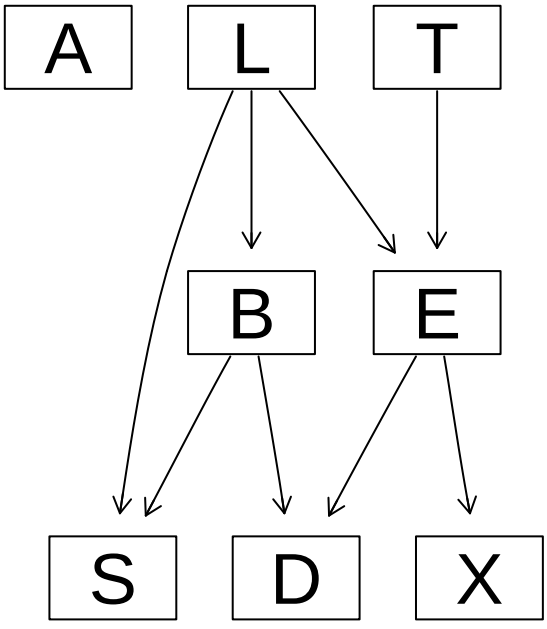

```
## [1] "Different number of directed/undirected arcs"
```






```
## [1] "Different number of directed/undirected arcs"
```





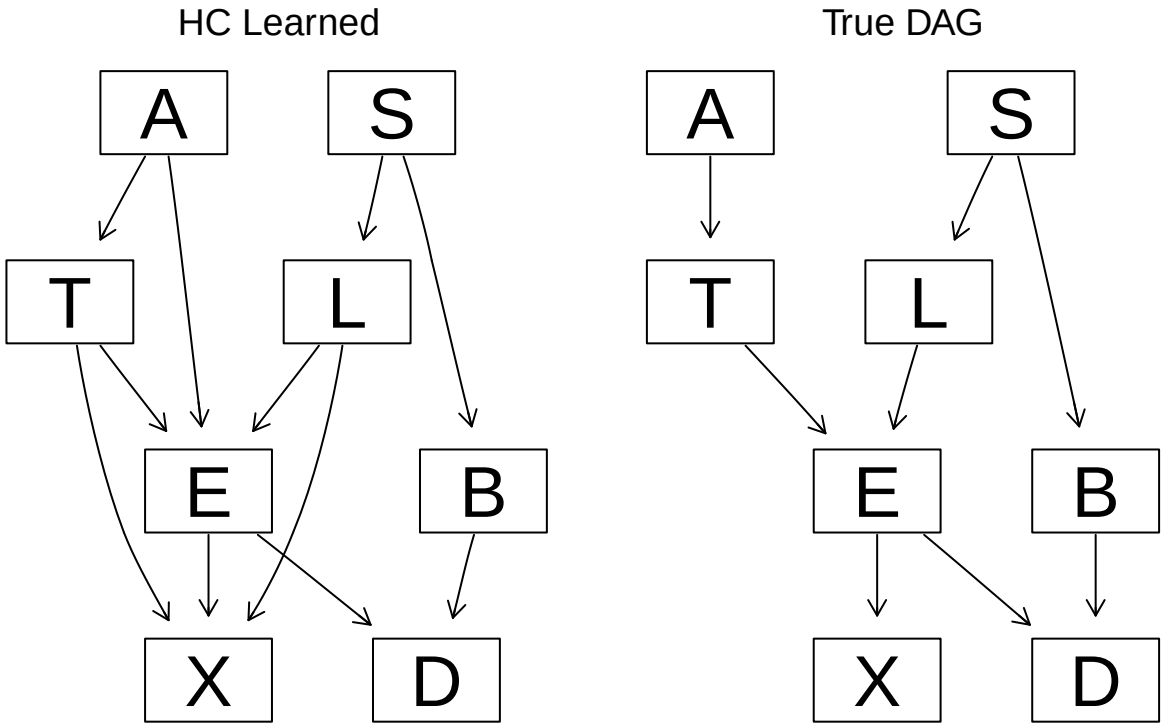

```
## [1] "Different number of directed/undirected arcs"
```

Answer. All three variants of hill climbing (different score function, different imaginary sample size, and different random starting structures; code in Appendix A.1) return structures that are not equivalent. This is shown by the plots and by converting each DAG to its equivalence class with `cpdag()` and comparing with `all.equal()`, which reports a different number of directed/undirected arcs in every case.

Hill climbing is a greedy local search: it repeatedly accepts the single edge change that gives the largest improvement in score and stops when no change improves it. Because the search is greedy it terminates at a local optimum, and which optimum it reaches depends on the starting structure and on the score being optimized. The imaginary sample size affects this too, since it controls the strength of the Dirichlet prior in the BDeu score and therefore how heavily dense structures are penalised.

Question 2

Learn a BN from 80 % of the Asia dataset. Use the BN learned to classify the remaining 20 % of the Asia dataset in two classes: S = yes and S = no.




```
##      hc_model_preds
##        no yes
##   no  337 176
##   yes 121 366

##      true_model_preds
##        no yes
##   no  337 176
##   yes 121 366
```

Answer. A network was learned from the 80 % training split using hill climbing with the BDE score and fitted by maximum likelihood (Appendix A.2). The two confusion matrices are identical, so the learned network and the true network classify every test case the same way, at an accuracy of 0.703.

The learned structure is not the true Asia network. What matters for prediction, however, is the posterior $P(S \mid \text{evidence})$, and this depends only on the Markov blanket of S. The two networks give S Markov blankets that carry the same information, so the posteriors, and hence the predictions, coincide. Structural differences elsewhere in the graph involve nodes that are conditionally independent of S given the blanket and therefore cannot affect the classification.

Question 3

Now classify S given observations only for the Markov blanket of S.

```
##      hc_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

```
##      true_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

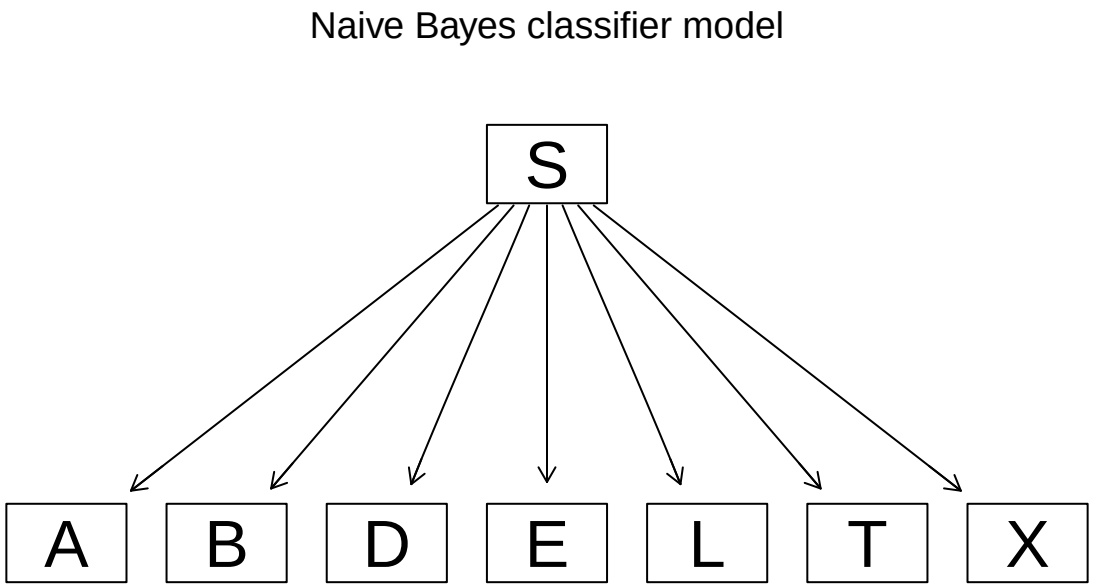
Answer. Restricting the evidence to the Markov blanket of S (Appendix A.3) reproduces the confusion matrices from Question 2 exactly. This is not a coincidence but follows from the definition of a Markov blanket: S is conditionally independent of every other variable given its blanket, so

$$P(S \mid A, T, L, B, E, X, D) = P(S \mid \mathrm{MB}(S)).$$

Observing the remaining variables cannot change the posterior. A practical consequence is that inference requires fewer evidence nodes at no cost in accuracy.

Question 4

Repeat exercise (2) using a naive Bayes classifier.




```
##      nbc_preds
##        no yes
##   no  359 154
##   yes 180 307
```

Answer. The naive Bayes network makes S the sole parent of every other variable, which encodes the assumption that the features are conditionally independent given S (Appendix A.4). The confusion matrix differs from Question 2 and accuracy falls to 0.666.

Question 5

Explain why you obtain the same or different results in the exercises (2–4).

Answer. Exercises 2 and 3 agree because the posterior $P(S \mid \text{evidence})$ is determined entirely by the Markov blanket of S. The learned and the true DAG differ in several arcs, but the local structure around S carries the same information, so both give the same posterior; and dropping the evidence that lies outside the blanket removes nothing that the posterior depends on. Recovering the correct graph is therefore not necessary for correct classification, only the blanket around the target variable matters.

Exercise 4 differs because the Naive Bayes classifier is making the assumption that all variables are conditionally independent This is a rather strong assumption that obviously does not hold given the known true DAG. In the Asia domain E is close to a deterministic function of T and L, X depends on the patient only through E, and D depends on B and E. These variables therefore carry largely overlapping evidence. Naive Bayes multiplies their likelihoods as though they were independent sources, which counts the same information several times and produces distorted, over-confident posteriors. The effect is visible in the error pattern and not only in the error rate: naive Bayes predicts "no" more often, gaining correct negatives but losing considerably more on false negatives.

Appendix A: R Code

A.1 Question 1: Hill climbing with variations

```r
# Different score functions
dag1 <- hc(asia, score = "bde")
dag2 <- hc(asia, score = "aic")
graphviz.plot(dag1, main = "BDE")
graphviz.plot(dag2, main = "AIC")
all.equal(cpdag(dag1), cpdag(dag2))

# Different imaginary sample sizes
dag3 <- hc(asia, score = "bde", iss = 5)
dag4 <- hc(asia, score = "bde", iss = 10)
all.equal(cpdag(dag3), cpdag(dag4))

# Different random starting structures
set.seed(12345)
dag5 <- hc(asia, score = "bde", start = random.graph(colnames(asia)))
dag6 <- hc(asia, score = "bde", start = random.graph(colnames(asia)))
all.equal(cpdag(dag5), cpdag(dag6))
```

A.2 Question 2: Learning, fitting and classifying

```r
# Split the data into 80% training and 20% test
n <- dim(asia)[1]
set.seed(12345)
id    <- sample(1:n, floor(n * 0.8))
train <- asia[id, ]
test  <- asia[-id, ]

# Learn a structure from the training data and compare with the true DAG
hc_dag   <- hc(train, score = "bde", iss = 5)
true_dag <- model2network("[A][S][T|A][L|S][B|S][D|B:E][E|T:L][X|E]")

# Fit parameters by maximum likelihood
hc_fit   <- bn.fit(hc_dag,   data = train, method = "mle")
true_fit <- bn.fit(true_dag, data = train, method = "mle")

# Convert to gRain objects for exact inference
hc_grain   <- as.grain(hc_fit)
true_grain <- as.grain(true_fit)

# Classify the test set: S = "yes" if P(S = yes | evidence) > 0.5, else "no"
classifier <- function(model, observed_nodes) {
  preds <- character(nrow(test))
  for (i in 1:nrow(test)) {
    states   <- as.matrix(test[i, observed_nodes])
    evidence <- setEvidence(model, nodes = observed_nodes, states = states)
    post     <- querygrain(evidence, nodes = "S")$S
```

```
    preds[i] <- ifelse(post["yes"] > 0.5, "yes", "no")
  }
  preds
}

nodes <- setdiff(colnames(asia), "S")

hc_model_preds   <- classifier(hc_grain,   nodes)
true_model_preds <- classifier(true_grain, nodes)
table(test$S, hc_model_preds)
table(test$S, true_model_preds)
```

A.3 Question 3: Classification with the Markov blanket only

```
hc_mb   <- mb(hc_fit,   node = "S")
true_mb <- mb(true_fit, node = "S")

hc_mb_preds   <- classifier(hc_grain,   hc_mb)
true_mb_preds <- classifier(true_grain, true_mb)
table(test$S, hc_mb_preds)
table(test$S, true_mb_preds)
```

A.4 Question 4: Naive Bayes classifier

```
# The naive Bayes classifier assumes that all feature variables are
# conditionally independent given the class variable S. As a Bayesian network
# this is a structure where S is the single parent of all feature nodes and
# no edges exist between the features.
nbc_network <- model2network("[S][A|S][T|S][L|S][B|S][E|S][X|S][D|S]")
graphviz.plot(nbc_network, main = "Naive Bayes classifier model")

nbc_fit   <- bn.fit(nbc_network, data = train, method = "mle")
nbc_grain <- as.grain(nbc_fit)
nbc_preds <- classifier(nbc_grain, nodes)
table(test$S, nbc_preds)
```